A Policy-Guided Reinforcement Learning Method for Encirclement Control in Multiobstacle Environment

Fandi Gou, Haikuo Du, Chenyu Zhao, and Yunze Cai

***Abstract*—The problem of multiagent encirclement with multiobstacle collision avoidance (EMOCA) has been challenging since it is difficult to balance the tradeoff between surrounding a mobile target and avoiding obstacles simultaneously. To address the EMOCA problem, we proposed a novel policy-guided reinforcement learning (RL) method, namely, multiregulator-assisted RL for encirclement control (MRA-RLEC) which leverages the jump-start learning and curriculum learning (CL) mechanism to enhance training efficiency. MRA-RLEC divides the complex encirclement task into a sequence of subtasks, progressively increasing in difficulty. In this process, multiple regulators are utilized to adjust various training aspects, including encirclement condition, obstacle avoidance, and the transition from guide to learned policy execution. Besides, a global encirclement reward decomposition (GERD) method is presented to alleviate reward sparsity, and we design a bidirectional communication protocol to reduce communication. Extensive experiments are carried out to showcase the robustness and superiority of our method, and the practical applicability of MRA-RLEC is demonstrated through experiments conducted in the robot operating system 2 (ROS2)-based simulation platform, Gazebo, using a self-designed omnidirectional vehicle model.**

***Index Terms*—Curriculum learning (CL), encirclement control, multiagent reinforcement learning (MARL), multiobstacle collision avoidance.**

I. Introduction

MULTIAGENT systems have become a focal point of research for their ability to outperform individual agents in tackling sophisticated challenges. They have extensive applications, including search and rescue [1], formation control [2], and pursuit-evasion games [3]. Among these, encirclement control has gained significant attention for its applicability in both civil and military fields. Encirclement control aims to surround an evading target within a predefined formation, with military applications focusing on target capture using UAVs or USVs [4], [5], as well as surveillance [6] and the escorting of high-value targets [7]. In civilian contexts, it aids in wildlife monitoring [8] and safeguarding ground vehicles [9]. As environments for encirclement become increasingly intricate, preventing collisions with obstacles is crucial for the safety of agents in complicated backgrounds such as dense forests or waters with reefs [10]. Due to the task complexity, the problem of multiagent encirclement with multiobstacle collision avoidance (EMOCA) has posed huge challenges.

Present encirclement control methods are broadly categorized into traditional and reinforcement learning (RL)-based approaches. Traditional approaches include control theory-based and optimization-based methods. In control theory-based methods [11], [12], [13], specific encirclement points are assigned to pursuers, and controllers are designed to drive the distances between pursuers and their target points to zero, thereby achieving encirclement. However, this method is reliant on agent kinematics and can be time-consuming. As for the optimization-based method, the encirclement process is modeled as an optimal control problem where model predictive control is deployed to minimize control effort and encirclement errors [4], [14], [15]. Despite its effectiveness, the computational demands of model predictive control are substantial, which may hinder real-time implementation.

With advancements in artificial intelligence, deep RL (DRL) has been widely applied to complex multiagent tasks [16]. In encirclement control, many studies have put forth DRL-based frameworks to address this challenge [17], [18], [19]. Ma et al. [17] proposed a multiagent deep deterministic policy gradient (MADDPG)-based strategy with a distributed control architecture using local information. However, it simplified scenarios by removing obstacles when encircling a moving target and assuming a static target in obstructed environments. Zhang et al. [18], [19] proposed graph attention networks to model spatial relationships between entities, enabling adaptation to varying numbers of robots and obstacles, but their experiments were limited to a small number of obstacles. Other works, such as [20] and [21], focused on multiagent surrounding problems within a bounded area, where no obstacle is considered and the terminal condition can be easily achieved with the help of environmental boundaries.

Received 13 June 2024; revised 22 December 2024 and 6 March 2025; accepted 27 April 2025. Date of publication 16 May 2025; date of current version 4 September 2025. This work was supported in part by the Open Project of the National Key Laboratory of Air-Based Information Perception and Fusion under Grant 202437 and in part by the Aeronautical Science Foundation of China under Grant 20220001057001 and Grant 20240001057002. *(Corresponding author: Yunze Cai.)*

Fandi Gou and Haikuo Du are with the Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, China, and also with the Key Laboratory of System Control and Information Processing, Ministry of Education of China, Shanghai 200240, China (e-mail: finley-gou3@sjtu.edu.cn; duhaikuo1013@sjtu.edu.cn).

Chenyu Zhao is with the Department of Electronic Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: pxwzq6@sjtu.edu.cn).

Yunze Cai are with the Department of Automation, Shanghai Jiao Tong University, Shanghai 200240, China, also with the Key Laboratory of System Control and Information Processing, Ministry of Education of China, Shanghai 200240, China, and also with the National Key Laboratory of Air-based Information Perception and Fusion, Luoyang 471000, China (e-mail: yzcai@sjtu.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2025.3566548

2162-237X © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

To tackle collision avoidance in multiobstacle environments, prevailing methods include artificial potential field (APF), Voronoi partition, off-line planning, and sense-and-avoid techniques [2]. Among them, APF is widely used in encirclement control due to its inherent symmetry and concise formulation [22], [23]. Voronoi partition-based approach also addresses pursuit-evasion and collision avoidance problems [24], [25]. Specifically, Wang et al. [25] proposed an encirclement algorithm by constructing an evader-centered bounded Voronoi cell (ECBVC), which cooperatively pursues an adversarial evader in an unbounded environment with cluttered obstacles. Nonetheless, both Voronoi partitioning and APF-based methods can encounter issues like local minima, control oscillation, and reliance on comprehensive environmental information, making them less effective in densely obstructed areas. The off-line planning method requires precomputing trajectories and is computationally expensive, especially in dynamic encirclement where targets are constantly moving [26]. Sense-and-avoid techniques are less common in encirclement due to their limited foresight and unreliable state prediction [27].

In DRL-based encirclement methods, most algorithms perform well only when obstacles are few, and the reasons are twofold. For one thing, the collision avoidance mechanism of DRL is to assign negative rewards upon collisions, while the existence of numerous obstacles can lead to prolonged periods of negative feedback and further cause training divergence. For another, the negative rewards from collisions near the terminal phase may counteract the positive rewards from successful encirclement, complicating the learning of the terminal state. Therefore, studies often address obstacle avoidance and encirclement in separate stages. From the above literature, it is evident that even though considerable studies have achieved satisfying encirclement results, it remains a challenge to balance the tradeoff between encircling a moving target and avoiding multiple obstacles.

Fortunately, innovative frameworks combining traditional and DRL methods have emerged to address environmental complexity in multiagent tasks. Sui et al. pioneered the integration of imitation learning with DRL for formation control [2], teaching the basic formation control in the imitation phase and refining them with RL in more complex scenarios. A subsequent study put forward a DRL approach combined with model-based paradigms [28], where they are merged in a parallel and a serial structure, respectively. These methods suggest the potential for tackling the complexities in moving target encirclement and obstacle avoidance.

Inspired by the abovementioned discussions, we propose a novel scheme to combine the model-based and DRL-based methods with the concept of jump-start RL (JSRL) [29]. We devise a guide policy-oriented technique with multiple curriculum learning (CL) regulators to address the challenge of EMOCA. Specifically, a model-based guide policy is introduced to assist training which significantly alleviates the issue of reward sparsity, and is applied to various multiagent RL (MARL) algorithms. Besides, we put forward a two-stage framework capable of adapting to more complex environments. We have also improved the design of the reward function and the communication protocol. The main contributions of our work are listed as follows.

1) A novel multiregulator-assisted RL for encirclement control (MRA-RLEC) scheme is proposed to tackle the encirclement problem of a moving target in an unbounded, multiobstacle environment. The training efficiency and convergence are improved through multiple regulators and the guide policy.
2) A global encirclement reward decomposition (GERD) method is presented to alleviate reward sparsity, encouraging the achievement of terminal encirclement condition. Moreover, a bidirectional communication protocol is designed to reduce communication load.
3) Comparative experiments are conducted to demonstrate the scalability and robustness of our method. Besides, the model is tested in Gazebo with robot operating system 2 (ROS2), based on a self-designed omnidirectional vehicle platform to verify the probability of real-world application.

The rest of this article is organized as follows. Section II gives the problem formulation of the encirclement scenario. Section III describes the preliminaries. Section IV elaborates the design procedure of the MRA-RLEC method, including guide policy, reward function, and network structure. Section V carries out simulations and experiments under different settings. Finally, Section VI concludes the article.

II. Problem Formulation

A. Encirclement Control Scenario

The encirclement control problem is considered within a 2-D unbounded space $\mathcal{W} \subseteq \mathbb{R}^2$ with M static obstacles, where a collective of N pursuers are required to encircle a mobile target while avoiding collision. The set of indices for the pursuers and obstacles are represented by $\mathcal{I} = \{1, 2, \ldots, N\}$ and $\mathcal{O} = \{1, 2, \ldots, M\}$, respectively, with individual pursuers and the target being referred to as P and T. For $i \in \mathcal{I}$, use $\mathcal{N}(i)$ to represent the neighbors of P_i. Suppose $i, j \in \mathcal{I}$, the included angle of P_i and P_j relative to the target T is defined as $\alpha_{i,j}$, and the distance between P_i and P_j is defined as $d_{i,j}$.

Definition 1 (Neighbors): When there are no other pursuers between $\alpha_{i,j}$, then P_i and P_j are defined as neighbors. Particularly, when the vector cross product $\overrightarrow{TP_i} \times \overrightarrow{TP_j} > 0$, P_j is deemed as the right neighbor of P_i, and the left neighbor is defined as the opposite. We define the direction perpendicular to the plane of $\mathcal{W}$ and pointing upward as the positive direction.

All the agents and obstacles are modeled in a circular form, where r^i and δ^i denote the radius and the safety buffer distance of each entity. At time t, the position and velocity of P_i are denoted as $p^i = [p^i_x, p^i_y]$ and $v^i = [v^i_x, v^i_y]$, $i \in \mathcal{I}$, while target's position and velocity are denoted as p^T and v^T. We use the superscript T to denote variables associated with the target in this article. Denote the obstacle i as O_i, $i \in \mathcal{O}$, and $O_i = [x^i_o, y^i_o, r^i_o, \delta^i_o]$. The dynamics of each agent are described in a second-order integrator form as

$$\begin{cases} \dot{p}^i = v^i \\ \dot{v}^i = u^i \end{cases} \tag{1}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

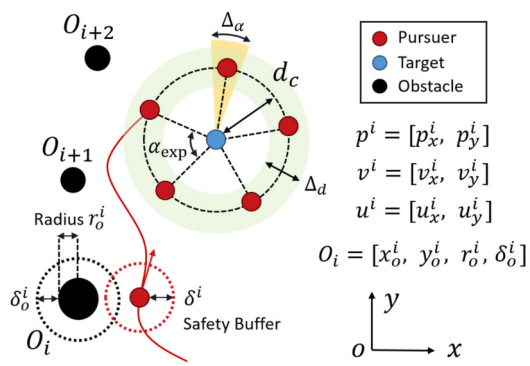


Fig. 1. Illustration of encirclement control scenario. The bands in light green and light yellow indicate distance bias and angle bias, respectively.

where $u^i = [u_x^i, u_y^i]$ denotes the acceleration of P_i, and each component of u^i has a limit $u_{\max}$. The same is true for the target. Besides, there are constraints in the maximum velocity for pursuers and the target, which means their velocity component cannot exceed $v_{\max}$ and $v_{\max}^T$, respectively. To guarantee the successful encirclement, we set $u_{\max} = u_{\max}^T, v_{\max} > v_{\max}^T$ in this article.

Initially, pursuers are aligned in a straight line with the target positioned randomly in a circular area ahead of them. Both pursuers and the target have zero initial speeds. For the encirclement condition, all the pursuers are supposed to be uniformly distributed around a circle centered by the target, with capture distance d_c as the radius. The expected encirclement angle for P_i with its neighbors is set to be $\alpha_{\exp} = (2\pi/N)$. To facilitate learning, a slight bias in the final encirclement angle Δ_α and distance Δ_d is permitted. The distance between P_i and T is denoted as $d_{i,T}$.

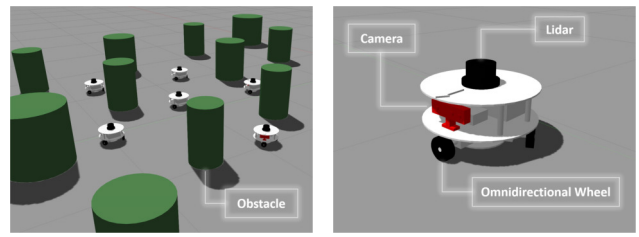
Definition 2 (Encirclement Condition): For any pursuer $i \in \mathcal{I}$, if the following conditions hold at the same time, the encirclement is considered as accomplished:

$$\begin{aligned} &|\alpha_{i,j} - \alpha_{\exp}| < \Delta_\alpha, \quad j \in \mathcal{N}(i) \\ &|d_{i,T} - d_c| < \Delta_d. \end{aligned} \tag{2}$$

Upon achieving the encirclement, all agents decelerate to static. A visual illustration of this encirclement condition is depicted in Fig. 1. Moreover, any pursuer cannot collide with other pursuers or obstacles and a safety buffer distance is necessary for the training. When the condition $\|p^i - p^j\| < r^i + r^j + k_\delta \times (\delta^i + \delta^j)$ is satisfied, where k_δ is a coefficient, the pursuers will get relevant punishment. The average distance between obstacles is set close to the capture distance d_c, making the encirclement scenario more complex.

B. ROS2-Based Simulation Platform

We utilize a high-fidelity autonomous navigation simulator, Gazebo, to construct a semi-physical experimental setting. A self-designed omnidirectional vehicle model platform is deployed for encirclement experiments, as shown in Fig. 2. Furthermore, we employ the open-source ROS2 to facilitate communication between our system and the Gazebo simulator [30]. To be noted, our algorithms are extendable to other platforms such as UAVs and USVs through kinematic adjustments.



(a) (b)

Fig. 2. Physical engine simulator Gazebo with ROS2. (a) Encirclement environment. (b) Self-designed omnidirectional vehicle model.

TABLE I
PARAMETERS OF OMNIDIRECTIONAL CAR

Feature	Description	Pursuer	Evader
m	Mass (kg)	0.8	0.8
r	Vehicle Radius (m)	0.15	0.15
$v_{\max}$	Max velocity (m/s)	0.5	0.25
$u_{\max}$	Max acceleration (m/s^2)	0.5	0.5
I_{xx}	Inertia moment $(kg \cdot m^2)$	0.006	0.006
I_{yy}	Inertia moment $(kg \cdot m^2)$	0.006	0.006

The inverse kinematic of the omnidirectional vehicle is calculated to obtain the speed of each wheel according to the desired velocity along each axis. Within the Gazebo simulation, the wheels are treated as rotational links, with their velocities being controlled by the simulator's inbuilt dynamics engine. The parameters of the model vehicle are shown in Table I.

III. PRELIMINARIES

A. Multiagent RL

MARL is an effective approach to solving sequential decision-making tasks for multiple agents, which can be modeled through a Markov decision process. This process is expressed by a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma \rangle$, where $\mathcal{S}$ is the state set and $s \in \mathcal{S}$ represents the global state of the environment. $\mathcal{A} = [\mathcal{A}_1, \ldots, \mathcal{A}_n]$ is the set of joint action and $\boldsymbol{a} \in \mathcal{A}$, where $\boldsymbol{a} = [a_1, \ldots, a_n]$. The component $a_i \in \mathcal{A}_i$ is the action taken by agent i $(i \in \{1, \ldots, n\})$. $\mathcal{T}: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ represents the state transition function and $R: \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ is the reward. $\gamma \in [0, 1]$ is the discount factor determining the importance of future rewards. The objective of the agents is to adopt a policy π that maximizes the expectation of discounted cumulative rewards

$$J(\pi) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t R^t \left(s^t, a_1^t, \ldots, a_n^t \right) \right] \tag{3}$$

where R^t is the reward that all agents obtain at time t.

Multiagent proximal policy optimization (MAPPO) is an extension of PPO in the multiagent scenarios [31], and is a policy gradient-based MARL method that utilizes the centralized training and decentralized execution framework [32]. The policy π is expressed as a parameterized deep neural network π^θ, where θ represents the parameters. The evaluation network is parameterized by ϕ. The parameters are updated by

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

maximizing the objective function $J(\pi^\theta)$ with gradient $\nabla_\theta J(\pi^\theta)$ as the optimization direction

$$\nabla_\theta J\left(\pi^\theta\right) = \nabla_\theta \log\left(\pi^\theta\left(a^t|s^t\right)\right)\sum_{t'=t}^{\infty}\gamma^{t'-t}R^{t'}\left(s^{t'},a^{t'}\right). \quad (4)$$

When updating parameters, the importance sampling ratio $l_t(\theta)$ is introduced to improve sample efficiency

$$l_t(\theta) = \frac{\pi^\theta\left(a^t|s^t\right)}{\pi^{\theta_{\text{old}}}\left(a^t|s^t\right)} \quad (5)$$

where $l_t(\theta)$ represents the likelihood ratio and θ_{old} is the policy parameter before the update. The loss function in the policy network is defined as follows:

$$L^\pi(\theta) = E\left[\min\left(l_t(\theta)\hat{A}_t^\theta\left(s^t,a^t\right),\right.\right.$$
$$\left.\left.\text{clip}\left(l_t(\theta),1-\epsilon,1+\epsilon\right)\hat{A}_t^\theta\left(s^t,a^t\right)\right)\right] \quad (6)$$

where $\hat{A}_t^\theta$ is the generalized advantage estimate and the clip function $\text{clip}(l_t(\theta),1-\epsilon,1+\epsilon)$ constraints the $l_t(\theta)$ within $[1-\epsilon,1+\epsilon]$ to avoid over update of the policy. The loss of value network is denoted by $L^V(\phi)$.

Compared with other MARL algorithms, MAPPO has superior performance in convergence and sampling efficiency. In this article, we select a variant of MAPPO with recurrent layers, rMAPPO, as the basic training algorithm. The details will be presented in Section IV.

B. Curriculum Learning

CL is inspired by the process of human learning, which progressively trains the model on tasks of increasing difficulty, starting from simpler to more complicated data. CL paradigm has a wide application scope in the realm of DRL [33], since it enhances sample efficiency, organizes model exploration, and solves sparse reward problems. Moreover, it contributes to accelerating the convergence of the learning process [34].

One specific application of CL in DRL is to control the data difficulty during the data collection, and a curriculum can be generally designed from the following aspects: initial states, reward functions, goals, environments, and opponents [35]. In the initial training stage, an easy curriculum is designed which simplifies the task to facilitate the acquisition of rewards. As the training advances, the task gradually restores to its original difficulty, thereby enhancing the model's adaptability to more challenging scenarios.

C. Jump-Start RL

Prior to the introduction of JSRL, the concept of base controllers had been utilized to facilitate training in the control of robotic arms [36]. Improving upon this concept, JSRL offers a compelling solution to address the sparse reward challenge encountered in RL. It initializes an RL policy with a preexisting policy and is compatible with any RL approach [29]. JSRL employs two policies to solve a specific task: a guide policy π^g and a learned policy π^θ. The guide policy is controlled by a curriculum, and is progressively withdrawn as the performance of learned policy increases. Both empirical and theoretical analyses have demonstrated that JSRL significantly outperforms the existing imitation learning algorithms due to its advantage in sample efficiency.

The mechanism of JSRL is described as follows. Let H symbolize the RL task horizon, and denote the model performance threshold as β_{thre}. Initially, select a sequence of guide-steps to roll out the guide policy, $(H_1,H_2,\ldots,H_n)$, where $H_i \in \{H\}$ denotes the guide-steps in the present iteration. Denote h as the iterator over the guide-step sequence $(H_1,H_2,\ldots,H_n)$. During the training, roll out π^g for h steps and π^θ for the remaining $H-h$ steps, which is $\pi_{1:h}=\pi^g$, $\pi_{h+1:H}=\pi^\theta$. After each parameter update, the model performance is evaluated through a function $V^*(\pi)$. When $V^*(\pi)>\beta_{\text{thre}}$, the iterator h advances to the subsequent value in the guide-step sequence. This process of training continues in a loop until the entire sequence of guide-steps is traversed.

IV. Policy-Oriented Learning for Encirclement Control

This section gives a detailed design procedure for MRA-RLEC and the MARL framework. The overall training architecture is depicted in Fig. 3. The training is divided into multiple threads, within each thread the trajectory data are collected and adjusted into a batch before optimizing parameters.

A. Guide Policy for JSRL

The APF is a classic path planning algorithm, which regards the target as an attractive source and the obstacles as repulsive sources, guiding the agent to move along the resultant force. However, the APF method has significant drawbacks as previously discussed in Section I. Nonetheless, APF is still able to drive the pursuers toward the target, which reduces the exploration space to some extent. In this regard, we employ APF as the guide policy for JSRL. To facilitate the formation of the desired encirclement and avoid local minima, we only account for three distinct forces: the repelling force between pursuers ($f_r^{i,j}$), between pursuers and obstacles ($f_o^{i,k}$), and the attraction force between pursuers and target ($f_a^{i,T}$), where $i,j\in\mathcal{I}$ and $k\in\mathcal{O}$. In the desired encirclement formation, the distance between neighboring pursuers is $L=2\,d_c\sin(\alpha_{\text{exp}}/2)$. The forces are generated as follows:

$$f_r^{i,j} = \begin{cases}\dfrac{k_r(L-d_{i,j})}{d_{i,j}}\left(p^i-p^j\right), & d_{i,j}\le L\\ 0, & d_{i,j}>L\end{cases} \quad (7)$$

$$f_o^{i,k} = \begin{cases}\dfrac{k_o(L_s-d_{i,k})}{d_{i,k}}\left(p^i-p^k\right), & d_{i,k}\le L_s\\ 0, & d_{i,k}>L_s\end{cases} \quad (8)$$

$$f_a^{i,T} = \frac{k_a(d_{i,T}-d_c)}{d_{i,T}}\left(p^T-p^i\right). \quad (9)$$

To alleviate vibration, a damping term containing v^i is added. The total force exerted on pursuer i is

$$u^i = \sum_{j=1,j\neq i}^{N} f_r^{i,j} + \sum_{k=1}^{M} f_o^{i,k} + f_a^{i,T} - k_b v^i \quad (10)$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

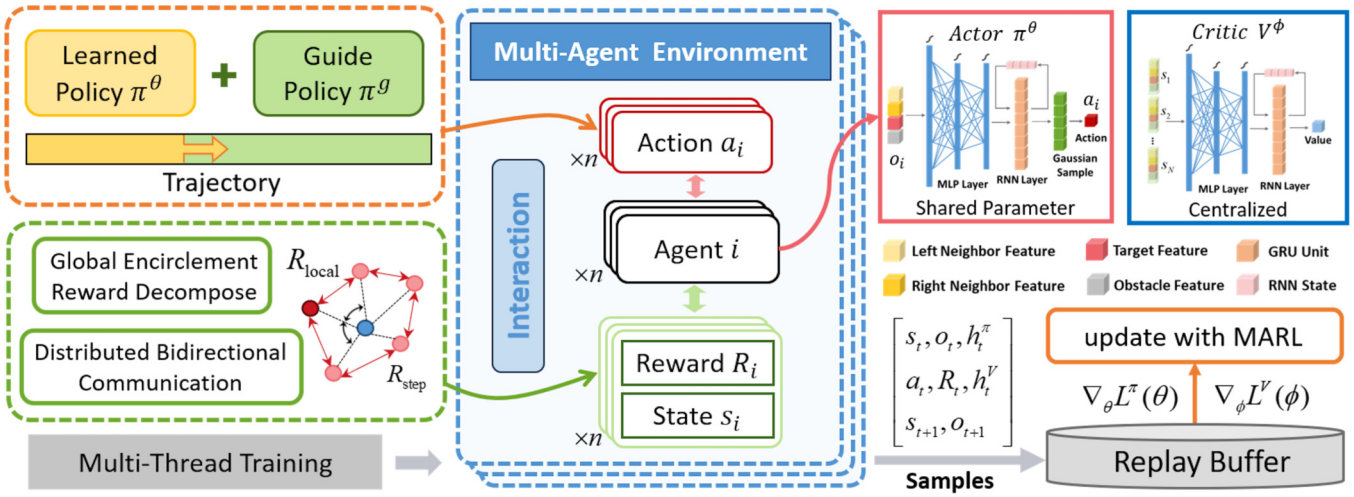


Fig. 3. Overall architecture of MRA-RLEC.

where k_r, k_o, k_a, and k_b are positive control gains, and L_s is a collision avoidance related threshold parameter.

As training advances, the initial steps of guide policy are gradually reduced, eventually giving way to the learned policy. In JSRL, the reduction of guide steps is based on a performance threshold β_{thre}, requiring the learned policy to meet specific criteria before reduction occurs. However, evaluating the policy performance brings uncertainty to the training, making it difficult to predict when the policy will meet the target level, potentially impacting subsequent training tasks. To address this, we introduce a nonlinear regulator to manage the guide policy's phase-out, ensuring a consistent, predetermined training duration for JSRL.

The designed regulator is intended to follow an S-shaped curve and here are the reasons. When the JSRL starts, pursuers require more space to explore under the guide policy, hence the slow reduction of guide-steps is necessary to encourage broader exploration. As the training progresses, the pursuers have identified a high reward terminal state, allowing for a more rapid reduction in guide-steps. However, near the end of JSRL, a slow retreat of the guide policy is essential due to the lasting influence of initial states on the future trajectory. If the learned policy has not fully grasped the optimal behavior around the initial states, abrupt reductions at this stage could impair the learning trajectory. Thus, the guide steps' reduction follows an S-shaped curve, ensuring a smooth transition from guided to learned policy execution.

To construct the S-shaped curve for the JSRL regulator, we select two odd functions as the foundation: $\tanh(x)$ and x^3. Since $\tanh(x)$ inherently has an S-shaped profile, the x^3 term is utilized to adjust the function value at the endpoints. To modulate the steepness of the S-curve, a scaling factor k_s is introduced into the variable x. Next, map both the horizontal and vertical coordinates to the range of 0–1 through function transformations. The final regulator function, which indicates the rate at which the guide policy steps are reduced, is formulated as follows:

$$f(x) = \frac{1}{2}\left(-\tanh(k_s X) - \varepsilon X^3 + 1\right) \tag{11}$$

where $X = 2x - 1$, $\varepsilon = 1 + \tanh(-k_s)$, and $f(\cdot)$ represents the JSRL regulator function.

Let E denote the total training episodes, $c_g \in (0, 1)$ is a ratio coefficient, $E_g = c_g E$ is defined as the training episodes of JSRL, and E_t denotes the current training episodes that have been completed. The ratio $E_t/E_g \in [0, 1]$ serves as an indicator of the progress made in the JSRL phase. Let M indicate the total steps in an episode, ρ_0 indicate the initial proportion of steps executed by guide policy, then the current number of guide-steps M^g is calculated by $M^g = M \times \rho_0 \times f(E_t/E_g)$. The remaining steps M^θ executed by the learned policy is given by $M^\theta = M - M^g$. When E_t/E_g reaches 1, the JSRL process is completed, and the training proceeds with the learned policy taking over all steps for the remainder of the training episodes.

B. Training Through Multiple Curriculum Regulators

Since EMOCA is a complex task, relying solely on JSRL to achieve encirclement is challenging. Given this, we implement a two-stage training strategy with a *Base* model trained in the first stage, and train a more complicated scenario in the *Fine Tune* stage. We divide the *Base* training process into multiple subtasks according to different goals. Each subtask is trained through CL during a certain process, from easy to hard. After that, we consider increased target speeds and various escape policies in the *Fine Tune* phase. The evolution of the entire training progress with multiple regulators is shown in Fig. 4.

There are four CL regulators in the training of *Base*, which are the JSRL regulator, angle regulator, distance regulator, and obstacle avoidance regulator. The JSRL regulator helps to reduce the exploration space, as mentioned previously. The angle regulator and distance regulator control the difficulty of terminal encirclement conditions by adjusting the angle bias and distance bias from large to small. This adjustment allows the pursuers to earn rewards early in the training process, even if they have not achieved the exact encirclement conditions, thereby mitigating the issue of sparse rewards. The training episodes of these two regulators are $E_e = c_e E$, and both

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

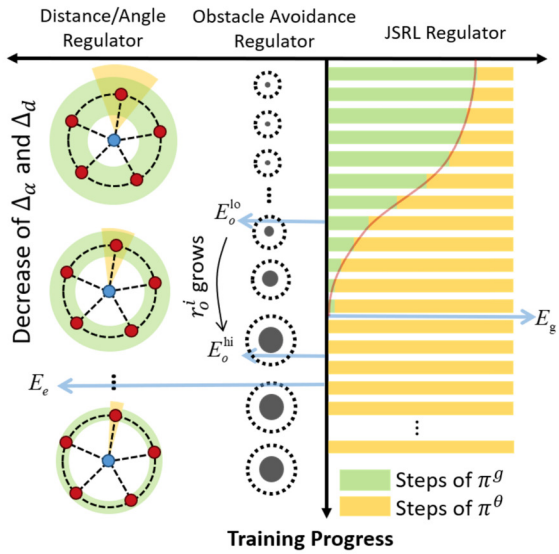


Fig. 4. Illustration of multiregulators that assist training. From left to right is the evolution of angle and distance bias, obstacle radius, and guide policy steps.

regulators are defined in linear form

$$\Delta_\alpha = \begin{cases} \Delta_\alpha^{\text{sta}} - \left(\Delta_\alpha^{\text{sta}} - \Delta_\alpha^{\text{exp}}\right) E_t/E_e, & E_t \le E_e \\ \Delta_\alpha^{\text{exp}}, & E_t > E_e \end{cases} \tag{12}$$

$$\Delta_d = \begin{cases} \Delta_d^{\text{sta}} - \left(\Delta_d^{\text{sta}} - \Delta_d^{\text{exp}}\right) E_t/E_e, & E_t \le E_e \\ \Delta_d^{\text{exp}}, & E_t > E_e \end{cases} \tag{13}$$

where the superscript "sta" indicates the start value, while "exp" indicates the expected value. Similar to c_g, the parameter c_e is a ratio that defines the proportion of episodes influenced by the regulator. The same applies to the later-mentioned parameters c^{lo} and c^{hi}.

As for the obstacle avoidance regulator, it modulates the sizes of obstacles within a specific training phase. Given that obstacles significantly influence the encirclement process, we introduce a mechanism where obstacles are not immediately presented at full size during training. Instead, they emerge incrementally, starting as mere points and progressively expanding to reach their original sizes over time. This process occurs between the training phase E_o^{lo} and E_o^{hi}, where $E_o^{\text{lo}} = c^{\text{lo}}E$ and $E_o^{\text{hi}} = c^{\text{hi}}E$. For $i \in \mathcal{O}$, the radius of obstacle i during this period is defined as follows:

$$r_o^i = \begin{cases} 0, & E_t < E_o^{\text{lo}} \\ r_o^{\text{exp}} \times \dfrac{E_t - E_o^{\text{lo}}}{E_o^{\text{hi}} - E_o^{\text{lo}}}, & E_o^{\text{lo}} \le E_t \le E_o^{\text{hi}} \\ r_o^{\text{exp}}, & E_t > E_o^{\text{hi}} \end{cases} \tag{14}$$

where r_o^{exp} is the original radius of obstacle i.

In the *Fine Tune* phase, the target escapes in different policies, with its speed gradually increasing through a linear curriculum regulator. Besides, the penalty for collision is raised, and the learning rate is set smaller. The comprehensive MRA-RLEC methodology is presented in Algorithm 1.

Algorithm 1 MRA-RLEC

Input: Total training episodes E, maximum episode steps M, learning rate l_r, PPO gradient update times K

Output: Actor network θ and critic network ϕ

Initialize actor parameter θ and critic parameter ϕ, define E_t as the current training episodes
for $E_t = 1$ **to** E **do**
  Compute E_t/E and select relevant regulators. Compute Δ_α and Δ_d by equation (12) and (13). Calculate r_o^i through (14) and set up encirclement scenario.
  Compute E_t/E_g and obtain guide step M^g via JSRL regulator function (11)
  for step = 1 **to** M **do**
    Execute the joint action $a_{1:N}^t = \pi(o_1, o_2, \ldots, o_N)$. If step $< M^g$, then set $\pi = \pi^g$, else set $\pi = \pi^\theta$
    Run the mixed policy of π^g and π^θ for M timesteps in total, collecting $\{o, a, r\}$ for each agent and obtain a trajectory τ. Store τ into replay buffer
  end for
  Estimate advantage function A_t from τ in replay buffer
  for k = 1 **to** K **do**
    Calculate policy loss $L^\pi(\theta)$ through equation (6)
    Update network parameters for each agent:
    $\theta' \leftarrow \theta - l_r \nabla_\theta L^\pi(\theta)$, $\phi' \leftarrow \phi - l_r \nabla_\phi L^V(\phi)$
  end for
end for

C. Design of MARL Framework

1) Reward Function: As for the EMOCA problem, it is difficult to describe the system reward only considering the final state, because the final state is hard to trigger which generally leads to sparse reward issues. To counter this, we propose the GERD method to allow rewards for partial encirclement, and the reward is designed as follows: before encirclement is finished, each pursuer receives a step reward R_{step} as an individual reward. To motivate local cooperation, pursuer i will receive a local surrounding reward R_{local} as long as it satisfies the condition in (2), at which point we consider pursuer i to be finished. Once finished, the pursuer keeps adjusting its position to maintain the partial encirclement and get continual rewards. This process continues until all pursuers are finished, ultimately leading to a complete encirclement. Upon the successful encirclement by all pursuers, each pursuer will receive a global capture reward R_{global}. The reward of pursuer i is defined as

$$R_i = \begin{cases} R_{\text{global}}, & \text{all pursuers are finished} \\ R_{\text{local}}, & \text{pursuer } i \text{ is finished} \\ R_{\text{step}}, & \text{otherwise.} \end{cases} \tag{15}$$

In the course of encirclement, the pursuer team needs to implement both surrounding and chasing actions simultaneously. Considering the symmetrical nature of the encirclement task, the sum of all the relative positional vectors between the pursuers and the target offers a natural representation of the encirclement state. Ideally, upon achieving a perfect encirclement, the magnitude of this aggregate vector should

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

converge to zero. Denote this as the formation reward R_f, which encourages both surrounding and chasing actions at the same time. To normalize this reward, R_f is defined as

$$R_f = \exp\left(-k_1 \cdot \| \sum_{i=1}^{N} \left(p^T - p^i\right) \| \right) - 1. \tag{16}$$

To ensure that the pursuers achieve the designated capture distance d_c, we introduce a distance reward R_d defined as follows:

$$R_d = \exp\left(-k_2 \cdot \sum_{i=1}^{N} \left(d_{i,T} - d_c\right)^2\right) - 1 \tag{17}$$

where k_1 and k_2 are tunable coefficients. Finally, we establish a fixed penalty for collisions, denoted as R_l. The step reward R_{step} is then formulated as a weighted sum of the formation reward R_f, the distance reward R_d, and the collision penalty R_l

$$R_{\text{step}} = w_1 R_f + w_2 R_d + w_3 R_l \tag{18}$$

where w_1, w_2, and w_3 are weights that determine the relative importance of each component in the overall reward. Specifically, we set $R_l = -3$ as the penalty for collisions, $R_{\text{local}} = 5$ as the reward for local cooperation, and $R_{\text{global}} = 10$ as the reward for successfully completing the encirclement. These values are chosen to create a balanced incentive structure that guides the learning process toward effective encirclement policies while avoiding collisions.

2) State Space and Communication Protocol: In most multiagent cooperative tasks, it is ideal for agents to communicate with all other agents to obtain complete information. However, in dynamic multiagent tasks such as encirclement, communicating with all agents is resource-consuming and puts forward higher requirements for communication bandwidth. Considering that the terminal encirclement state is a standard circle, we propose a distributed bidirectional communication protocol, so that each pursuer only exchanges information with its left neighbor and right neighbor.

Define S_t^i as the state vector for pursuer i at time t, and its neighbors are pursuer $j, k \in \mathcal{N}(i)$. S_t^i consists of four distinct components, that are $S_t^{i,j}$, $S_t^{i,k}$, $S_t^{i,T}$, and $S_t^{i,o}$. Specifically, $S_t^{i,j}$ and $S_t^{i,k}$ represent the relative states of pursuer i to its two neighbors. For instance, the state relative to its left neighbor P_j is given by $S_t^{i,j} = \left[\alpha_{i,j}, d_{i,j}, v_{i,j}, Q_{i,j}, Q_{j,i}\right]^\top$, where $v_{i,j}$ is the relative velocity's magnitude, and $Q_{i,j}$ is the relative bearing angles from P_i's velocity to the position vector pointing from P_i to P_j, as illustrated in Fig. 5. The right neighbor state $S_t^{i,k}$ has the same components. The term $S_t^{i,T}$ represents the observed state of target, defined as $S_t^{i,T} = \left[d_{i,T}, \dot{d}_{i,T}, Q_{i,T}, \dot{Q}_{i,T}, \omega, \dot{\omega}\right]^\top$, where $Q_{i,T}$ is the bearing angle between pursuer's velocity and the relative position vector, while ω denotes the target's rotational velocity from the perspective of pursuer i. The observation of the closest obstacle o is captured by $S_t^{i,o} = \left[x_{i,o}, y_{i,o}, d_{i,o}, d_{j,o}, d_{k,o}, r_o, v_x^i, v_y^i\right]^\top$, with $x_{i,o}$ and $y_{i,o}$ reflecting the positional differences along the x- and y-axes between pursuer i and

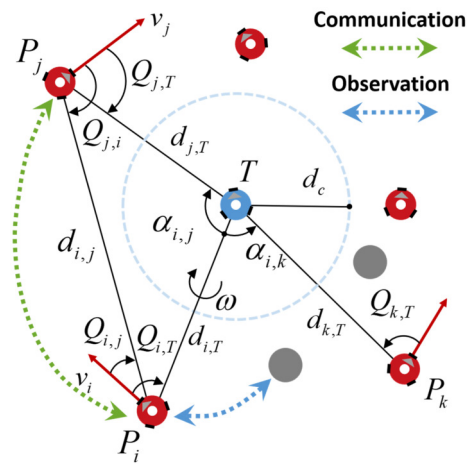


Fig. 5. Schematic of encirclement features.

obstacle o, respectively. Finally, a concatenated state S_t^i is derived

$$S_t^i = \left[\left(S_t^{i,j}\right)^\top, \left(S_t^{i,k}\right)^\top, \left(S_t^{i,T}\right)^\top, \left(S_t^{i,o}\right)^\top\right]^\top \tag{19}$$

and S_t^i is the immediate input to the actor network.

3) Network Structure: Given that MAPPO is under actor-critic architecture, each pursuer is equipped with an actor network that maps the observed states into its actions, and the whole system has a centralized critic network that evaluates the overall state. Both actor network and critic network are based on fully connected and recurrent neural network (RNN) structure, where hidden states $h^{\pi,V}$ in RNN restore historical information and promote decision-making. The fully connected structure is constructed using a multilayer perceptron (MLP), and the gate recurrent unit is employed as the RNN layer to capture temporal dependencies. Moreover, $\tanh(\cdot)$ is selected as the activation function, and all the actor networks have the same structure and share the model parameters.

V. Simulations and Results

In this section, we apply the MRA-RLEC to different MARL algorithms to verify its robustness and effectiveness, and we train the model with different numbers of agents to show the scalability of the proposed method. Without loss of generality, we choose the default number of agents as five, i.e., $N = 5$. Besides, we compare MRA-RLEC with other encirclement algorithms to validate its efficacy. We also conduct ablation tests and demonstrate the practicality of our method through experiments in Gazebo using self-designed omnidirectional vehicles.

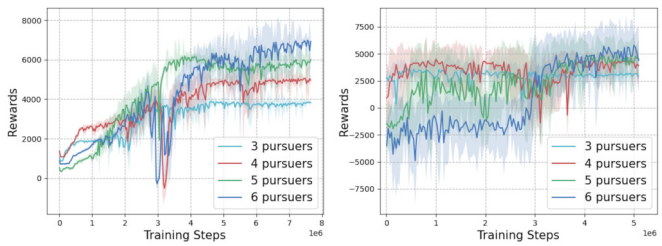
A. Simulation Settings

In the training process of *Base*, we set the regulator parameters c_g, c_e, c^{lo}, and c^{hi} to 0.6, 0.75, 0.45, and 0.75, respectively. The discount factor γ is set to 0.985, and the clip coefficient ϵ is set to 0.15. The weights of step reward are set to $\omega_1 = 0.3, \omega_2 = 0.3$, and $\omega_3 = 0.4$. In terms of pursuit dynamics, the maximum velocity of pursuer $v_{\max}$ is set

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

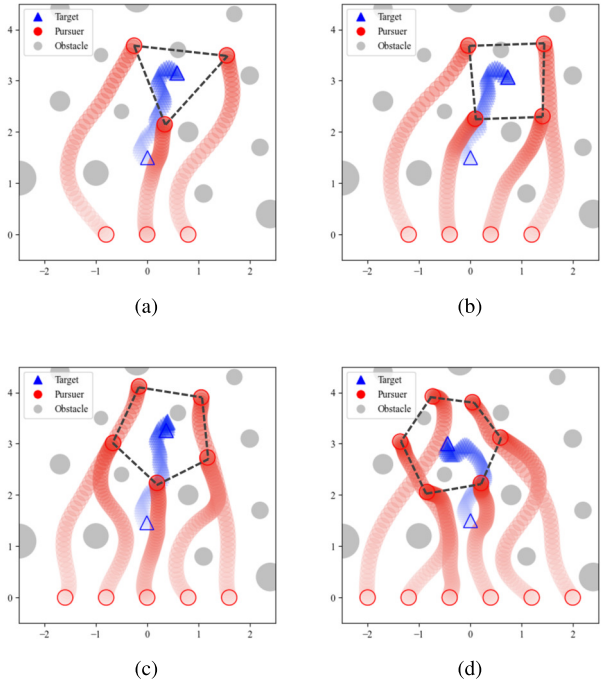
TABLE II
NETWORK AND TRAINING RELATED PARAMETERS

Name	Parameter
Actor network MLP layers	2
Actor network hidden units	128, 64
Actor network RNN hidden units	32
Critic network MLP layers	3
Critic network hidden units	256, 128, 64
Critic network RNN hidden units	32
Length of RNN data chunk	10
Updating batch size	100
Learning rate in *Base*	0.0002
Learning rate in *Fine Tune*	0.00015
PPO update epoch	15



(a) (b)

Fig. 6. Training rewards of different numbers of pursuers. (a) Reward of *Base*. (b) Reward of *Fine Tune*.



(a) (b) (c) (d)

Fig. 7. Trajectories of different numbers of pursuers via MRA-RLEC, trained with rMAPPO. The axis unit is meters (m). (a) 3 pursuers encirclement. (b) 4 pursuers encirclement. (c) 5 pursuers encirclement. (d) 6 pursuers encirclement.

to 0.5 m/s, while $v_{\max}^T$ is set to 0.15 m/s in the *Base* and 0.25 m/s in *Fine Tune*. The acceleration limits, represented by $u_{\max}$, are set to 0.5 m/s^2 for both sides.

For encirclement condition parameters, we set $d_c = 1$ m, $\Delta_\alpha = 0.3$ rad, and $\Delta_d = 0.1$ m. We also set the parameters in the guide policy as $k_r = 4.5, k_o = 4.0$, $k_a = 2.0$, and $k_b = 1.5$, and the collision distance threshold is $L_s = 0.3$ m. Furthermore, the maximum steps of each episode are set to 200, and each simulation time interval dt represents 0.1 s. The *Base* phase is designed to undergo a total of 7.68×10^6 training steps, while the *Fine Tune* phase comprises 5.12×10^6 steps. In addition, the experiment is applied on a computer with an Intel Xeon E5-2678 v3 CPU, 128 GB of memory, and Nvidia GTX 1080Ti GPU. Other network and training-related parameters are illustrated in Table II.

B. Scalability and Robustness of MRA-RLEC

To verify the scalability of MRA-RLEC, we implement our algorithm on the environments presented in Section II with numbers of pursuers varying from 3 to 6. During the training of *Base*, we set the JSRL parameter $k_s = 2.0$. In the *Fine Tune* stage, we increase the maximum target speed and use a linear CL regulator to gradually increase the speed. The potential field policy (PFP) [37] is selected as the escape policy for the target, and experiments are conducted under the rMAPPO scheme.

The visualizations of rewards in the two stages are shown in Fig. 6. As seen from Fig. 6(a), in *Base* the reward curves usually experience a drop before going upward. This phenomenon can be attributed to the retreat of the guide policy, which can lead to a decrease in training rewards. While in *Fine Tune*, there is no guide policy, and the reward steadily increases. The convergence of reward in Fig. 6(a) and (b) indicates that MRA-RLEC is scalable to different numbers of pursuers.

The model is further validated, with the trajectories of the encircling process visualized in Fig. 7. The radius of all the entities is consistent with the simulation parameters, and the trajectory colors vary in depth to indicate the chronological sequence of movements. Fig. 7 illustrates the successful encirclement formation, where the pursuers form a regular polygon centered by the target, satisfying $|\alpha_{i,j} - \alpha_{\exp}| < \Delta_\alpha$, $j \in \mathcal{N}(i)$ and $|d_{i,T} - d_c| < \Delta_d$. Besides, any apparent overlap in the trajectories of the pursuers is a result of differences in timing and not due to physical collisions. The figure also confirms that there is no intersection between the trajectories and the obstacles, indicating that no collisions occur between the agents and the obstacles during the encircling process.

To verify the robustness of MRA-RLEC, we apply it to various MARL algorithms to train a five-pursuer encirclement scenario. Among these algorithms, the independent proximal policy optimization (IPPO) method provides a scheme where each agent estimates its local value function independently [38]. MAPPO has no RNN layers compared with rMAPPO, and in multiagent transformer (MATransformer) the networks use transformer structure rather than convolutional neural networks [39]. In addition, heterogeneous-agent PPO (HAPPO) [40], MADDPG [32], and multiagent twin delayed DDPG (MATD3) [41] are all prominent algorithms in the domain of MARL. The reward visualization is shown in Fig. 8, and by all methods the training rewards converge properly under our framework.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

TABLE III
REWARD INDICES WITH MRA-RLEC BASED ON DIFFERENT MARL ALGORITHMS

Training method	Average reward (↑)	Training steps to achieve average reward (↓)	Maximum reward (↑)	Average epoch standard deviation(↓)	Standard deviation of average reward(↓)
rMAPPO	**4345.177**	2630400	6390.158	599.2639	1908.99
MAPPO	2870.418	2598400	5941.362	567.4647	1942.78
IPPO	4141.887	2342400	6372.418	727.4509	1790.40
HAPPO	3366.252	**1862400**	5518.821	612.6576	1533.79
MATransformer	4290.802	2374400	6312.289	864.2967	**1304.26**
MADDPG	4017.395	2438400	6836.482	405.1290	2414.49
MATD3	3977.205	2790400	**6848.849**	**252.7119**	2658.52

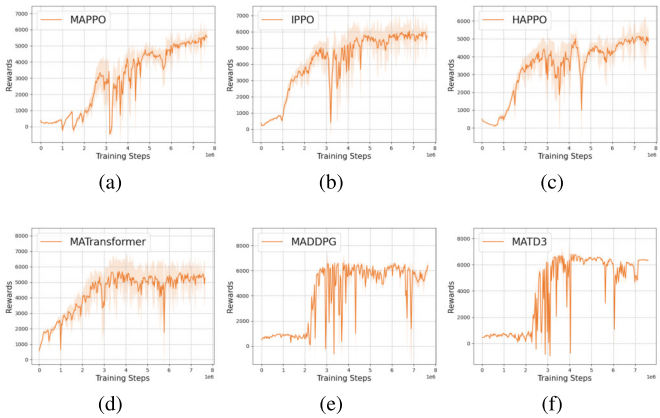


Fig. 8. Training rewards of different MARL algorithms trained with MRA-RLEC. (a) MAPPO. (b) IPPO. (c) HAPPO. (d) MATransformer. (e) MADDPG. (f) MATD3.

Table III compares training metrics of different algorithms, utilizing ↑ for higher desirable values and ↓ for the opposite. The epoch standard deviation, indicating reward variance in an epoch, is shown in the shaded area under the reward curve. The standard deviation of the average reward across all epochs is a measurement of reward fluctuation, which reflects the policy stability during training. Among these methodologies, off-policy algorithms (MADDPG and MATD3) distinguish between target policy and behavior policy, and are more likely to explore the global optimal policy. This can lead to more reward oscillation since they explore more policies. Therefore, off-policy algorithms have higher maximum reward and standard deviation in average reward. In comparison, PPO-based algorithms are on-policy methods, which may settle for local optima. Furthermore, the presence of RNN layers, as seen in the contrast between MAPPO and rMAPPO, significantly affects average rewards.

C. Ablation Test

We first conduct an ablation test to demonstrate the effectiveness of the JSRL regulator. In this experiment, a five-pursuers encirclement scenario is focused, and the JSRL regulator is introduced in the training process. The initial ratio of guide policy steps ρ_0 is set to 0.8. By varying the JSRL parameter k_s, we generate a spectrum of regulator functions, each of which is utilized to train the model.

As seen from Fig. 9, the training reward achieves convergence when the proportion curve exhibits a convex shape in the initial stage, where $k_s > 1$. Nevertheless, the proportion curve cannot be overly convex, as this could cause an abrupt withdrawal of the guide policy in the middle, thereby affecting the exploration. This is well illustrated by the reward curves $k_s = 2.5$ and $k_s = 3.0$. Conversely, when the proportion curve is concave, with $k_s < 1$, the model's convergence performance deteriorates. This aligns with the analysis that the model requires additional time to explore the terminal condition in the initial stage. Besides, we thoroughly withdraw the guide policy after a few episodes of training to demonstrate the influence of JSRL, and the gray curve shows that the rewards cannot converge in such a complex scenario.

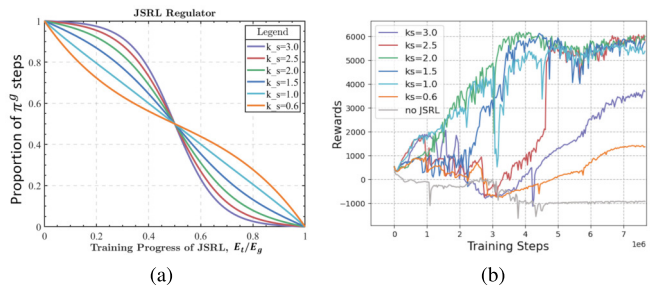


Fig. 9. Ablation test of JSRL regulators. (a) Proportion curves of guide policy steps in different k_s. (b) Training rewards under different JSRL regulators.

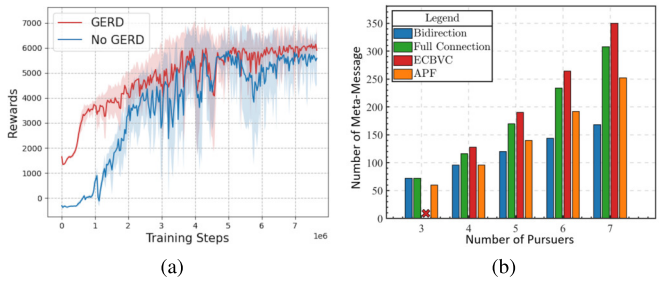


Fig. 10. Comparative results in reward function and communication design. (a) Training reward with and without GERD. (b) Meta-messages cost in different encirclement algorithms.

Next, we test the effectiveness of GERD. To simplify the scenario, we remove obstacles and assist in training with the JSRL regulator. The first experiment incorporated GERD, granting local rewards for partial encirclement. In the comparative experiment, rewards are given only when complete encirclement is formed. As illustrated from Fig. 10(a), the introduction of GERD accelerates the increase in rewards and leads to a higher convergence value. In contrast, the model

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

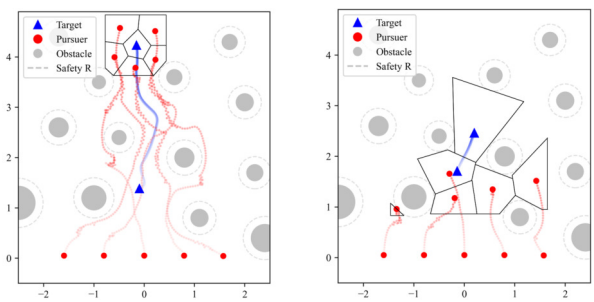


Fig. 11. Encirclement trajectories via ECBVC. (a) Successful encirclement. (b) Unsuccessful encirclement.

trained without GERD spends more time exploring for full encirclement, yielding lower initial rewards. This comparison highlights GERD's role in enhancing training efficiency and convergence.

We further evaluate the communication efficiency of our bidirectional protocol using the meta-message [20], which could measure the algorithms' communication load complexity in a single timestep. Define N_p, N_T, and N_o as the observation dimensions for pursuers, the target, and obstacles, respectively. The total communication meta-message under the bidirectional protocol is calculated as $(2N_p+N_T+N_o)N$, the load complexity is $\mathbb{O}(N)$. While in the fully connected situation, the communication load is $((N-1)N_p+N_T+N_o)N$, and the complexity is $\mathbb{O}(N^2)$. The ECBVC and APF methods require each pursuer to have full knowledge of all entities, resulting in meta-message scales with the square of N, the load complexity is also $\mathbb{O}(N^2)$. The comparison result is shown in Fig. 10(b), with the ECBVC algorithm lacking support for scenarios $N = 3$. This analysis underscores the efficiency of our bidirectional protocol for conserving communication resources in encirclement tasks.

D. Comparison With Other Encirclement Algorithms

In this section, we compare MRA-RLEC with other encirclement algorithms. We utilize rMAPPO, MADDPG, and MATD3 as fundamental RL frameworks and integrate MRA-RLEC on these training bases, with APF serving as the guide policy. In comparison, we have selected three typical traditional approaches: ECBVC [25], pseudo-force-based encirclement control (PFEC) [23], and formation-based encirclement control (FBEC) [11]. The performance is evaluated by the following key metrics: average step reward, encirclement time, danger rate, and successful rate. Danger rate represents the frequency of situations when two entities are too close, and such situation is determined by safety buffer, specifically when $\|p^i - p^j\| < r^i + r^j + k_\delta \times (\delta^i + \delta^j)$.

An encirclement test is deemed unsuccessful by either failing to meet the encirclement criteria or colliding with allies or obstacles. Fig. 11 illustrates the instances of successful and unsuccessful encirclements under ECBVC, where the cells in black lines represent Voronoi cells. The trajectory indicates the center of the pursuer and target, while the buffer distance around the obstacle corresponds to the radius of the pursuer. In Fig. 11(b), due to the close distance between the obstacle and pursuer, the left pursuer is trapped in the local extremum within its Voronoi cell. This leads to an oscillation in its control, ultimately resulting in a failed encirclement attempt.

To further evaluate the generality of different encirclement algorithms, we have designated different evasion policies for the target, as outlined below.

1) The PFP considers the coalition threats of the pursuer team and helps the target act responsively. The corresponding control law v_1^T is

$$v_1^T = \left\|v_{\max}^T\right\| \frac{\sum_i \frac{\hat{d}_{i,T}}{\|p^T - p^i\|}\left(p^T - p^i\right)}{\left\|\sum_i \frac{\hat{d}_{i,T}}{\|p^T - p^i\|}\left(p^T - p^i\right)\right\|} \tag{20}$$

where $\hat{d}_{i,T} = d_{i,T} - (r^T + r^i)$, and the entity radius are involved to compute the potential forces.

2) The greedy policy (GP) drives the target away from the nearest pursuer g. The corresponding control law v_2^T is

$$v_2^T = \left\|v_{\max}^T\right\| \frac{p^T - p^g}{\|p^T - p^g\|}. \tag{21}$$

3) The Apollonius policy (AP) [42] helps the target stay within its dominance region and maximizes the capture time for pursuers. The corresponding control law v_3^T is

$$v_3^T = \left\|v_{\max}^T\right\| \frac{p^A - p^T}{\|p^A - p^T\|} \tag{22}$$

where p^A is the farthest boundary point of the closest Apollonius circles among all target-pursuer pairs.

4) The move-to-centroid policy (MCP) [25] keeps the target away from neighboring pursuers by moving toward its Voronoi centroid. The corresponding control law v_4^T is

$$v_4^T = \left\|v_{\max}^T\right\| \frac{C_{\mathcal{V}_T^{\mathcal{W}}} - p^T}{\left\|C_{\mathcal{V}_T^{\mathcal{W}}} - p^T\right\|} \tag{23}$$

where $C_{\mathcal{V}_T^{\mathcal{W}}}$ is the center of target's Voronoi cell.

We performed a Monte Carlo simulation, running each algorithm 100 times, with the results presented in Table IV. The first column represents different escape policies, while the second corresponds to different encirclement algorithms. As indicated in Table IV, MADDPG and MATD3 combined with MRA-RLEC have the best performance in encirclement time, owing to their capability in finding globally optimal solutions. For RL-based methods, the faster pursuers form encirclement formation, the greater terminal rewards they accumulate, resulting in a smaller encircle time. In terms of danger rate and successful rate, RL-based methods generally have better performance than traditional ones. Because in traditional methods, the history information is merely considered, and the control laws greatly rely on the positions of obstacles, which could be less effective in a complex multiobstacle environment. As a result, much time is spent on avoiding collisions, leading to a poor performance in encirclement.

Another comparative dimension is the computational complexity. With our bidirectional protocol and a fixed network structure, each pursuer only processes information from two neighbors and the most threatening obstacle. Hence, the forward inference complexity remains constant from the view

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
ENCIRCLEMENT INDICES WITH DIFFERENT METHODS UNDER VARIOUS ESCAPE POLICIES

Escape Policy	Encircle Method	Step Reward	Encircle Time (s)	Danger Rate (%)	Success Rate (%)
PFP	rMAPPO	**21.44**	9.79±0.43	**7**	**100**
	MADDPG	19.57	10.61±0.98	16	99
	MATD3	21.28	**9.65±0.66**	12	**100**
	ECBVC	8.73	13.07±1.23	42	28
	PFEC	9.04	13.58±2.21	79	41
	FBEC	10.69	14.33±1.51	60	47
GP	rMAPPO	**24.48**	8.89±0.11	11	**100**
	MADDPG	23.65	9.21±0.59	15	**100**
	MATD3	22.97	**8.74±0.32**	**9**	**100**
	ECBVC	7.29	12.38±1.12	49	26
	PFEC	13.25	12.20±2.58	58	67
	FBEC	11.99	13.51±3.97	70	32
AP	rMAPPO	21.85	9.81±0.17	12	99
	MADDPG	22.29	9.52±0.38	7	**100**
	MATD3	**23.91**	**8.66±0.49**	**4**	**100**
	ECBVC	8.83	13.21±1.96	45	43
	PFEC	9.82	13.54±3.08	79	52
	FBEC	8.80	14.57±1.86	62	42
MCP	rMAPPO	25.43	8.13±0.11	8	**100**
	MADDPG	**26.78**	**7.45±0.20**	6	**100**
	MATD3	26.39	7.93±0.09	**5**	**100**
	ECBVC	7.87	11.58±1.15	39	37
	PFEC	10.77	11.96±2.45	58	74
	FBEC	13.79	13.23±1.68	34	72

of information. In contrast, for other methods that require fully connected communication, each pursuer has to integrate information from all other entities, resulting in a computational complexity of $\mathbb{O}(NM)$. From this perspective, our approach demonstrates a significant advantage in terms of efficiency.

E. Performance Analysis

In addition, we have conducted experiments to validate the performance of our algorithm under different levels of task complexity, and analyzed the effectiveness of balancing between encircle a mobile target and avoid obstacles. The task complexity is determined by the maximum target speed, and we select a series of typical velocities to perform the Monte Carlo test, with each parameter conducting 1000 trials. Denoted by S and R, the success rate and the average step reward are documented, as detailed in Table V.

As observed from Table V, there is a decline in performance when $v_{\max}^T$ exceeds a critical threshold. This decrement is attributed to the increased difficulty for pursuers to catch up with a faster-escaping target. Consequently, the model prioritizes maximizing the encirclement reward, potentially at the expense of obstacle avoidance, to obtain a higher overall return. Such behavior reflects the intrinsic tradeoffs in RL-based methodologies, where the learned policy adapts to maximize overall task performance. Therefore, the algorithm achieves its best effectiveness when the task is simple, and could give poor performance under highly complicated conditions. In extreme scenarios, particularly when $v_{\max}^T$ is nearly equal to $v_{\max}$, pursuers are unable to catch up with the target, making encircle it even more unattainable.

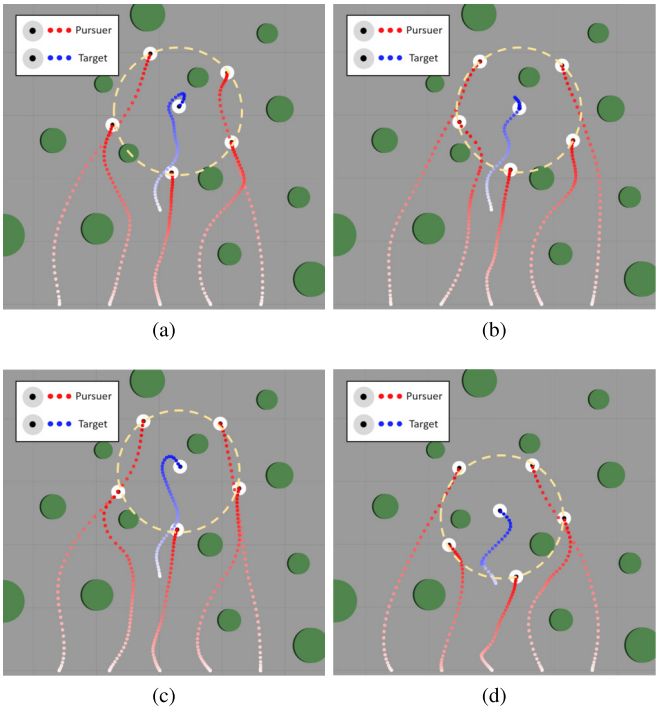


Fig. 12. Encirclement trajectories under different escape policies running in ROS2, trained via MRA-RLEC framework and rMAPPO. (a) Escape with PFP. (b) Escape with GP. (c) Escape with AP. (d) Escape with MCP.

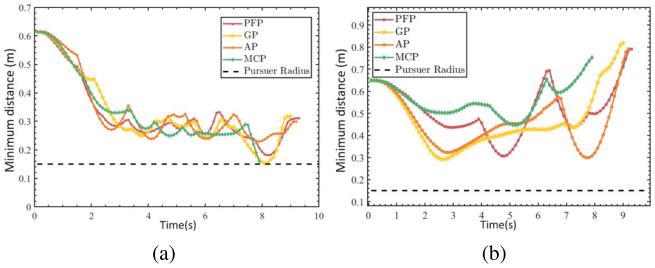


Fig. 13. Distance to evaluate collisions when encircling the target. (a) Minimum distance between pursuers and obstacles $D_o(t)$. (b) Minimum distance among all pursuers $D_p(t)$.

F. Experiments Under Various Escape Policies in ROS2

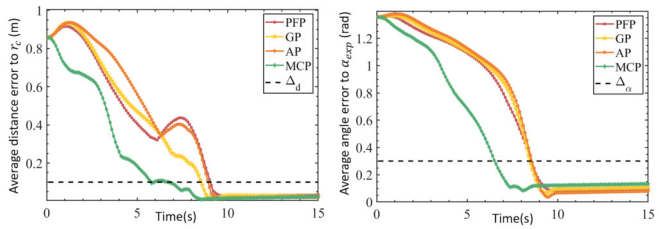
For future real-world vehicle deployment, we conducted additional experiments in Gazebo with ROS2 managing vehicle control and communication. The model, trained using MRA-RLEC combined with rMAPPO, generates accelerations from the actor network. The wheel velocities are further calculated through kinematics and the control commands are published via the ROS2 system. Vehicles communicate by publishing and subscribing to states through a shared message pool. We performed a five-pursuer encirclement test with varied target escape policies, as depicted in Fig. 12, where trajectory lines in red and blue indicate the sequence of movements over time.

In addition, visualizing the minimum distances helps evaluate collision avoidance effectiveness. D_o is defined as the minimum distance from any pursuer to an obstacle, calculated

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PERFORMANCE UNDER DIFFERENT ENCIRCLEMENT COMPLEXITY

Method	$v_{\max}^T = 0.10$ m/s		$v_{\max}^T = 0.20$ m/s		$v_{\max}^T = 0.25$ m/s		$v_{\max}^T = 0.27$ m/s		$v_{\max}^T = 0.30$ m/s	
	S (%)	R	S (%)	R	S (%)	R	S (%)	R	S (%)	R
MADDPG	99.7	36.74	99.2	23.53	99.1	20.24	59.8	9.59	31.4	3.87
MATD3	99.8	35.97	99.5	25.08	99.3	21.17	63.2	11.82	29.8	3.07
RMAPPO	99.8	39.48	99.3	25.76	98.7	20.65	65.7	13.37	34.2	4.16



(a) (b)

Fig. 14. Distance error and angle error under four escape policies over time. (a) Distance error e_d, calculated by average distance minus capture radius. (b) Angle error e_α, calculated by average included angle minus expected encirclement angle.

as $D_o = \min_{i\in\mathcal{I}, j\in\mathcal{O}} (d_{i,j} - r_o^j)$, where r_o^j is the radius of the obstacle j. Fig. 13(a) illustrates that D_o is always above the pursuer's radius, signifying effective obstacle avoidance. Fig. 13(b) presents the minimum distances between pursuers as $D_p = \min_{i,j\in\mathcal{I}, i\neq j} (d_{i,j} - r^j)$, with r^j as P_j's radius, confirming no collisions occur among pursuers during encirclement.

Fig. 14 illustrates the errors associated with the encirclement condition. The average distance error of pursuers, denoted by $e_d = \frac{1}{N}\sum_{i\in\mathcal{I}} |d_{i,T} - d_c|$, gradually descends below Δ_d and converges to zero, as shown in Fig. 14(a). Similarly, the average angle error of expected encirclement formation, denoted by $e_\alpha = \frac{1}{N}\sum_{i=1}^{N} |\alpha_{i,i+1} - \alpha_{\exp}|$, also descends below Δ_α and converges properly.

VI. CONCLUSION AND FUTURE WORK

This article presents a novel policy-guided RL framework, MRA-RLEC, to tackle the problem of mobile target encirclement in a multiobstacle environment. In MRA-RLEC, multiple regulators are devised to assist training, and the GERD method is proposed to alleviate reward sparsity. Moreover, we designed a resource-saving bidirectional protocol which has a linear communication complexity as the number of pursuers increases. We tested the scalability and robustness of MRA-RLEC with different pursuer numbers and MARL structures, and compared our method with other encirclement algorithms, demonstrating our superiority. Along with this, we also conducted experiments under various escape policies in the physical simulation engine Gazebo to demonstrate MRA-RLEC's potential for real-world application.

While our experimental outcomes are favorable within the defined settings, certain limitations still exist. Some factors present in real-world systems are not fully accounted for in our model such as communication range, and the target moves at a relatively low speed. In future work, we intend to address these challenges by conducting real-world experiments with highly maneuverable targets that have broader sensing ranges. Besides, it is worth noting that the concept of guide policy and curriculum regulators is extendable to other multiagent tasks. This framework could potentially improve training efficiency in any multiagent system that adopts the RL approach.

REFERENCES

[1] M. Ashish, J. Vilela, G. Nejat, and B. Benhabib, "A multirobot path-planning strategy for autonomous wilderness search and rescue," *IEEE Trans. Cybern.*, vol. 45, no. 9, pp. 1784–1797, Sep. 2014.

[2] Z. Sui, Z. Pu, J. Yi, and S. Wu, "Formation control with collision avoidance through deep reinforcement learning using model-guided demonstration," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 6, pp. 2358–2372, Jun. 2021.

[3] R. Zhang, Q. Zong, X. Zhang, L. Dou, and B. Tian, "Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7900–7909, Sep. 2022.

[4] A. T. Hafez, M. Iskandarani, S. N. Givigi, S. Yousefi, and A. Beaulieu, "UAVs in formation and dynamic encirclement via model predictive control," *IFAC Proc. Volumes*, vol. 47, no. 3, pp. 1241–1246, 2014.

[5] X. Qu, W. Gan, D. Song, and L. Zhou, "Pursuit-evasion game strategy of USV based on deep reinforcement learning in complex multi-obstacle environment," *Ocean Eng.*, vol. 273, Apr. 2023, Art. no. 114016.

[6] A. Sarwal, D. Agrawal, and S. Chaudhary, "Surveillance in an open environment by co-operative tracking amongst sensor enabled robots," in *Proc. Int. Conf. Inf. Acquisition*, Jul. 2007, pp. 345–349.

[7] G. Antonelli, F. Arrichiello, and S. Chiaverini, "The Entrapment/Escorting mission," *IEEE Robot. Autom. Mag.*, vol. 15, no. 1, pp. 22–29, Mar. 2008.

[8] A. Khan, B. Rinner, and A. Cavallaro, "Cooperative robots to observe moving targets: Review," *IEEE Trans. Cybern.*, vol. 48, no. 1, pp. 187–198, Jan. 2018.

[9] F. Chen, W. Ren, and Y. Cao, "Surrounding control in cooperative agent networks," *Syst. Control Lett.*, vol. 59, no. 11, pp. 704–712, Nov. 2010.

[10] Z. Sun, H. Sun, P. Li, and J. Zou, "Cooperative strategy for pursuit-evasion problem with collision avoidance," *Ocean Eng.*, vol. 266, Dec. 2022, Art. no. 112742.

[11] N. Huang, J. Zhuge, Z. Sun, D. Huang, Y. Kong, and Q. Lu, "Dynamic encirclement for anonymous agents with arbitrary formations," *IEEE Trans. Control Netw. Syst.*, vol. 11, no. 3, pp. 1644–1654, Sep. 2024.

[12] B. Xu, H.-T. Zhang, Y. Ding, and W. Ren, "Event-triggered surrounding formation control of multiagent systems for multiple dynamic targets," *IEEE Trans. Control Netw. Syst.*, vol. 10, no. 2, pp. 752–764, Jun. 2023.

[13] F. Liu, S. Yuan, W. Meng, R. Su, and L. Xie, "Multiple noncooperative targets encirclement by relative distance-based positioning and neural antisynchronization control," *IEEE Trans. Ind. Electron.*, vol. 71, no. 2, pp. 1675–1685, Feb. 2024.

[14] A. T. Hafez, A. J. Marasco, S. N. Givigi, M. Iskandarani, S. Yousefi, and C. A. Rabbath, "Solving multi-UAV dynamic encirclement via model predictive control," *IEEE Trans. Control Syst. Technol.*, vol. 23, no. 6, pp. 2251–2265, Nov. 2015.

[15] J. Liao, C. Liu, and H. H. T. Liu, "Model predictive control for cooperative hunting in obstacle rich and dynamic environments," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2021, pp. 5089–5095.

[16] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, Sep. 2020.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.

[17] J. Ma, H. Lu, J. Xiao, Z. Zeng, and Z. Zheng, "Multi-robot target encirclement control with collision avoidance via deep reinforcement learning," *J. Intell. Robotic Syst.*, vol. 99, no. 2, pp. 371–386, Aug. 2020.

[18] T. Zhang, Z. Liu, Z. Pu, and J. Yi, "Multi-target encirclement with collision avoidance via deep reinforcement learning using relational graphs," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 8794–8800.

[19] T. Zhang, Z. Liu, S. Wu, Z. Pu, and J. Yi, "Multi-robot cooperative target encirclement through learning distributed transferable policy," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–8.

[20] Y. Wang, L. Dong, and C. Sun, "Cooperative control for multi-player pursuit-evasion games with reinforcement learning," *Neurocomputing*, vol. 412, pp. 101–114, Oct. 2020.

[21] C. de Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulic, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 4552–4559, Jul. 2021.

[22] W. M. Spears, D. F. Spears, J. C. Hamann, and R. Heil, "Distributed, physics-based control of swarms of vehicles," *Auton. Robots*, vol. 17, nos. 2–3, pp. 137–162, 2004.

[23] F. Liu, S. Yuan, K. Cao, W. Meng, and L. Xie, "Distance-based multiple noncooperative ground target encirclement for complex environments," *IEEE Trans. Control Syst. Technol.*, vol. 33, no. 1, pp. 261–273, Jan. 2025.

[24] B. Tian, P. Li, H. Lu, Q. Zong, and L. He, "Distributed pursuit of an evader with collision and obstacle avoidance," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13512–13520, Dec. 2021.

[25] X. Wang, L. Xi, Y. Ding, and B. M. Chen, "Distributed encirclement and capture of multiple pursuers with collision avoidance," *IEEE Trans. Ind. Electron.*, vol. 71, no. 7, pp. 7520–7530, Jul. 2024.

[26] A. Richards and J. P. How, "Aircraft trajectory planning with collision avoidance using mixed integer linear programming," in *Proc. Amer. Control Conf.*, vol. 3, May 2002, pp. 1936–1941.

[27] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2017, pp. 285–292.

[28] Z. Pu, T. Zhang, X. Ai, T. Qiu, and J. Yi, "A deep reinforcement learning approach combined with model-based paradigms for multiagent formation control with collision avoidance," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 7, pp. 4189–4204, Jul. 2023.

[29] I. Uchendu et al., "Jump-start reinforcement learning," in *Proc. 40th Int. Conf. Mach. Learn.*, Jan. 2022, pp. 34556–34583.

[30] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Sci. Robot.*, vol. 7, no. 66, p. 6074, May 2022.

[31] C. Yu, A. Velu, E. Vinitsky, Y. Wang, A. M. Bayen, and Y. Wu, "The surprising effectiveness of PPO in cooperative, multi-agent games," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Jan. 2021, pp. 24611–24624.

[32] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Jan. 2017, pp. 1–12.

[33] X. Wang, Y. Chen, and W. Zhu, "A survey on curriculum learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 9, pp. 4555–4576, Sep. 2022.

[34] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *J. Mach. Learn. Res.*, vol. 21, no. 1, pp. 1–50, Jul. 2020.

[35] R. Portelas, C. Colas, L. Weng, K. Hofmann, and P.-Y. Oudeyer, "Automatic curriculum learning for deep RL: A short survey," 2020, *arXiv:2003.04664*.

[36] G. Wang, M. Xin, W. Wu, Z. Liu, and H. Wang, "Learning of long-horizon sparse-reward robotic manipulator tasks with base controllers," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 4072–4081, Mar. 2022.

[37] X. Fang, C. Wang, L. Xie, and J. Chen, "Cooperative pursuit with multi-pursuer and one faster free-moving evader," *IEEE Trans. Cybern.*, vol. 52, no. 3, pp. 1405–1414, Mar. 2020.

[38] C. Schroeder de Witt et al., "Is independent learning all you need in the StarCraft multi-agent challenge?," 2020, *arXiv:2011.09533*.

[39] M. Wen et al., "Multi-agent reinforcement learning is a sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Jan. 2022, pp. 16509–16521.

[40] Y. Zhong, J. G. Kuba, S. Hu, J. Ji, and Y. Yang, "Heterogeneous-agent reinforcement learning," *J. Mach. Learn. Res.*, vol. 25, pp. 1–67, Jan. 2023.

[41] J. Ackermann, V. Gabler, T. Osa, and M. Sugiyama, "Reducing overestimation bias in multi-agent domains using double centralized critics," 2019, *arXiv:1910.01465*.

[42] A. Von Moll, D. W. Casbeer, E. Garcia, and D. Milutinovic, "Pursuit-evasion of an evader by multiple pursuers," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2018, pp. 133–142.

Fandi Gou received the B.Eng. degree in automation from Shanghai Jiao Tong University, Shanghai, China, in 2023, where he is currently pursuing the Ph.D. degree in control science and engineering with the Department of Automation.

He is the first Chair-Elect of the Chinese Association of Automation Student Chapter with Shanghai Jiao Tong University. His research interests include multiagent reinforcement learning, encirclement control, and cooperative coverage.

Haikuo Du received the B.Eng. degree in automation from Shanghai Jiao Tong University, Shanghai, China, in 2023, where he is currently pursuing the master's degree in control science and engineering with the Department of Automation.

His current research interests include path planning, obstacle avoidance control, and safe reinforcement learning.

Chenyu Zhao is currently pursuing the B.Eng. degree with the Department of Electronic Engineering, Shanghai Jiao Tong University, Shanghai, China.

Her current research interests include multiagent cooperation and reinforcement learning.

Yunze Cai received the B.S. degree from the Department of Applied Mathematics, Northwestern Polytechnical University, Xi'an, China, in 1997, the M.S. degree from the Department of Automation, Northwestern Polytechnical University, in 2000, and the Ph.D. degree from the Department of Automation, Shanghai Jiao Tong University, Shanghai, China, in 2003.

She is currently a Professor with the Department of Automation, Shanghai Jiao Tong University, and a Visiting Researcher at the National Key Laboratory of Air-based Information Perception and Fusion. She has been engaged in the research of information processing of complex systems, and the application of intelligent decision-making and control in multi-agent systems.

Prof. Cai is a member of the Chinese Association of Automation, the Information Fusion branch of the Chinese Society of Aeronautics and Astronautics, and the Chinese Artificial Intelligence Association.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on April 04,2026 at 14:27:15 UTC from IEEE Xplore. Restrictions apply.